

Stock Reorder Agent Project Review

15 September 2026 | Current implementation and MSME product direction

Assessment: The project is a useful prototype for identifying stock shortages and preparing supplier purchase orders. Its sample workflow works, and its small, readable codebase is a good starting point. It is not ready for routine production purchasing or a general-purpose Stocklist product: inventory tracking is incomplete, and verified defects affect approval, order identity, data accuracy and repeat use.

1 What the project does

An operator supplies an Excel inventory snapshot. The application finds items below their reorder levels, groups them by supplier, drafts one purchase-order email per supplier, and lets the operator review, edit, send or skip it. Successful live sends can be recorded in a CSV log. The immediate value for a micro or small business is reducing manual shortage checks and repetitive email preparation.

Current flow: Excel file → stock rules → supplier groups → purchase-order draft → operator decision → Gmail email → CSV send record.

2 How the codebase is organized

File	Responsibility and relationship
streamlit_app.py	Web entry point. Loads inventory, displays summary metrics and supplier tabs, edits email text, and coordinates send status. Calls the business tools directly.
agent.py	Separate command-line entry point. Uses Groq to choose tool calls and draft emails; manages terminal approval and send/log guards.
tools.py	Shared implementation: Excel parsing, shortage calculations, terminal preview, Gmail delivery, CSV logging and rupee formatting.
po_utils.py	Deterministic web helpers: supplier grouping, date-based PO numbers and plain-text email templates.
tool_schemas.py	Five model-facing function definitions describing the tools and their expected arguments. These are not a public business API.
config.py	Reads environment settings for Groq, Gmail, business identity, file paths and dry-run mode.
setup_sample.py	Creates an eight-item demonstration workbook and prints expected outcomes. Running it overwrites inventory.xlsx in the working directory.

Important distinction: The web application does not invoke the AI agent or require Groq to calculate shortages and draft emails. Only the command-line workflow uses Groq. The README combines these capabilities, which can give an inaccurate impression of the web architecture.

Review coverage: all 12 original project files, including 7 Python files totaling 1,110 lines, README, dependencies, ignore rules, Excel data and CSV log. This is a downloaded source snapshot without Git history. Code references below refer to these local files.

3 Data and end to end behavior

Inventory contract. Each row represents an item with item_name, item_code, current_stock, reorder_level, reorder_qty, unit, supplier_name, supplier_email, unit_price and optional category. The supplied workbook has one sheet and eight rows. The reader uses the first worksheet, recognizes common English headers such as SKU, Vendor and Qty on Hand, and converts four numeric columns. Extra columns can survive parsing, but they do not add new business behavior. No mandatory-field or uniqueness rules are enforced. *[tools.py:55-115]*

Reordering rules

An item qualifies only when **current stock < reorder level**; equality does not trigger an order. Shortage equals reorder level minus current stock. A positive reorder quantity from the file is used as supplied. Otherwise, the fallback is **1.5 × reorder level – current stock**, rounded to two decimals. Line value equals order quantity × unit price. For example, stock 2 and threshold 10 give a fallback order of 13 units, leaving 15 after receipt. *[tools.py:122-176]*

This is a fixed threshold rule, not demand forecasting. It does not consider supplier lead time, pending purchase orders, customer commitments, pack sizes, minimum purchase values or purchasing budgets. Units are labels: a box-to-piece or kilogram-to-gram conversion is not calculated. The importer also treats “safety stock” as a reorder level and “min order qty” as a fixed order quantity, although those concepts need separate meanings in a general inventory product.

Verified sample outcome

Supplier	Low-stock items	Order value
City Safety Supplies	1	₹5,000
Rathi Packaging Co	2	₹12,600
Sharma Stationery	2	₹26,000
Total	5 of 8 inventory items	₹43,600

Web and command line execution

Web: Upload an .xlsx file or use the configured workbook. Four metrics show inventory rows, low-stock rows, suppliers needing orders, and proposed order value. Supplier tabs show item tables and editable subject/body fields. Dry-run is the default; live sending also requires the sidebar confirmation. Session-only statuses are pending, dry_run, sent, skipped and send_failed. Item quantities and prices have no structured editor, so email edits do not recalculate the displayed or logged totals. *[streamlit_app.py:54-174]*

Command line: Groq receives the workbook data through tool results, drafts emails and requests terminal approval. The operator can approve, skip or edit. A maximum of 50 model turns limits the loop. Local functions perform file and email operations; the model decides their sequence. This path adds an external data dependency and model-generated arguments to an otherwise deterministic workflow. *[agent.py:139-263]*

What is stored and what happens next

Emails are plain text, sent through Gmail over SSL, with a STARTTLS fallback for SMTP exceptions. The CSV stores timestamp, PO number, supplier name/email, item count and total value; it does not store order lines, approved text, delivery state or receipts. Its three supplied rows total ₹43,600, but do not independently prove delivery. Sending a PO never updates inventory. There is no receipt, sale, return, transfer or stock-adjustment ledger, and no tracking of supplier replies. *[tools.py:260-326]*

4 Quality and verified risks

What is good: The code separates configuration, shared tools and web helpers; the web workflow is understandable; dry-run is enabled by default; the AI loop has an actual pre-send approval guard; and the sample arithmetic is correct. These are useful foundations. Production readiness is held back by the following issues.

High priority 1 — A draft can belong to the wrong inventory. Web draft fields and statuses use only date/sequence PO numbers as keys. After switching to another inventory in a test, the supplier tab changed but the earlier supplier's subject and body remained. Sending could therefore address a new recipient with an old order. Bind each draft to a persistent supplier ID and inventory/order version. *[streamlit_app.py:100-133]*

High priority 2 — Approval does not lock the content or prevent resend. The AI guard checks only recipient and subject, not the approved body. A mocked model sequence sent changed content twice after one approval. The web and CLI also regenerate the same date/sequence IDs on a new run, without consulting order history. Persist unique orders and make each approved version sendable only once. *[agent.py:166-251; po_utils.py:11-13]*

High priority 3 — The live-send flag leaves logging in dry-run mode. With the default settings, --no-dry-run changes config.DRY_RUN and tools.DRY_RUN but leaves agent.DRY_RUN true. Delivery can be live while the banner says dry-run and the agent skips logging. The flag mismatch was reproduced. Use a single explicit run configuration. *[agent.py:31-34, 151, 221-227, 286-291]*

High priority 4 — Invalid inventory can look valid. A text cell containing "1,000" became stock zero; "1,250.50" became price zero. A missing reorder column returned no low-stock items, and a missing supplier column crashed the web grouping step. Supplier groups with the same name but different emails were merged into the first address. Validate required fields, numeric formats, unique item/supplier identities and recipient consistency before drafting. *[tools.py:99-111; po_utils.py:16-26]*

High priority 5 — Amounts and email content can disagree. The rupee formatter truncates decimals: ₹123.99 displays as ₹123. Email formatting also cuts long item codes and quantity strings. Free-form email edits leave the calculated totals unchanged. Preserve decimal precision and generate the final email and log from the same validated order lines. *[tools.py:29-35; po_utils.py:40-48; streamlit_app.py:123-145]*

Before shared deployment — Users are not isolated. Uploaded files share one temporary filename, send mode is a process-wide variable, and the CSV has no locking. There are no implemented user accounts, roles or business-level data boundaries. Concurrent users could overwrite inputs or interfere with state. These are source-level risks; concurrent load was not tested. *[streamlit_app.py:41-45, 132; tools.py:309-323]*

Reliability and recovery need work. A dry-run leaves that PO's send button disabled even after enabling live mode, which was reproduced. Failed/skipped orders also lack a reset path. Email and CSV writes are separate, so a send can succeed before logging fails. The AI path can raise uncaught errors for missing tool arguments; it has no application-level recovery or durable resume mechanism. *[streamlit_app.py:126-159; agent.py:115-132, 169-177]*

Engineering status: The snapshot contains no automated test suite, CI workflow, dependency lockfile, database migrations, backup/restore process or monitoring. README refers to a missing .env.example. Every web interaction rereads the workbook; the CLI repeatedly includes growing inventory/tool history in model requests. Large inventories and hosted persistence remain unverified.

Validation: All seven Python files parsed. Streamlit 1.59.0 / Python 3.12 tests confirmed the sample metrics, draft behavior and documented defects. External AI and SMTP actions were mocked or dry-run; no real supplier emails were sent. Live delivery, Groq behavior and deployment reliability are not established by these checks. Original files remained unchanged.

5 Building a generalized MSME Stocklist

Build a shared inventory core with purchasing attached. Add optional industry modules so traders, workshops and food businesses can use different details and controls.

First — Stabilize ordering. Fix defects and add validated imports with a column-mapping preview, persistent unique PO numbers, exact approvals and send/recovery controls. Save order lines and approved content before delivery. Add regression tests, tested dependencies and clear setup instructions.

Next — Maintain stock throughout the business cycle. Add item and supplier masters, searchable stock lists, barcode lookup, warehouses/bins and controlled units of measure. Record opening balances, goods receipts, sales/issues, returns, transfers, damage and physical stock counts in a transaction ledger. Track on-hand, reserved, available and incoming quantities separately. Support partial receipts and the PO lifecycle from draft and approval through receipt and closure. A ledger is the standard foundation for explaining each quantity change. [1]

Then — Make daily use practical for MSMEs. Add owner, purchaser, storekeeper and accountant roles; business-level isolation; mobile-friendly stock entry; Excel/CSV import and export; backup and restore; and a history showing who changed what. Include reorder alerts, overdue purchases, inventory valuation, slow/dead stock and supplier performance reports. Offer local-language labels and saved import mappings. For Indian deployments, plan configurable GST/HSN fields and accounting exports, with applicable tax behavior validated before release.

Improve purchase decisions after transaction data is reliable. Use lead times, consumption history, safety stock, open orders, reserved demand, minimum quantities, pack multiples and available budget. Explain each recommendation and allow an audited override. Considering incoming orders and committed demand avoids replenishing solely against an old stock snapshot. [2] Forecasting and an AI assistant should come after this foundation.

Industry features should be optional

MSME segment	Optional extension
Retail and distribution	Size/color variants, multiple selling units, price lists, customer orders and point-of-sale integration.
Food and other perishables	Batches, expiry dates, first-expiring-first-out picking, waste and recall traceability.
Manufacturing and workshops	Bills of materials, raw materials, work in progress, finished goods, production consumption and job work.
Equipment and spare parts	Serial numbers, alternate parts, warranty and service/repair history.

Recommended architecture and delivery gates

Keep one modular application with shared business rules and a transactional database. Store businesses, users, items, suppliers, locations, stock movements, POs/lines, approvals and delivery attempts. Use a durable email queue with explicit handling of uncertain delivery. AI may explain or draft; code should own quantities, money, permissions and state changes. Variants, batches and serial tracking are established item-model extensions. [3]

Rollout gates: Prove that invalid data cannot produce orders, repeated actions cannot duplicate sends, and receipts/issues/returns reconcile to stock balances. Pilot with one business after partial-receipt handling, role checks, backup restoration and failure recovery pass. Expand industry coverage using observed needs.

Sources: [1] ERPNext — Stock Ledger [2] ERPNext — Projected Quantity [3] Zoho Inventory — Items.

Official documentation checked 15 September 2026. Priorities are engineering recommendations; these capabilities are not yet implemented.